Práctica de estrategias algorítmicas: Backtracking

Alejandro García García, Marcelo Fort Muñoz

Contenido

1-Introducción 1

2-Explicación componentes de s 1

3-Funciones 1

4-Presentación y análisis de resultados de número de pasos en las funciones sin usadas y con usadas 2

1-Introducción

En esta práctica implementamos en C una estrategia algorítmica para la resolución de un problema computacional en el que se precisa encontrar la asignación óptima de un conjunto de tareas a un grupo de personas. La estrategia en cuestión es el método de "*backtracking".*

La técnica de vuelta atrás o "*backtracking"* consiste en realizar una búsqueda exhaustiva y sistemática en el espacio de soluciones, lo que suele ser bastante ineficiente. Esta búsqueda se realiza entre todas las combinaciones posibles de recursos, construyendo primero una solución parcial y comprobando si, a partir de esta, podemos llegar a una solución final. En el caso de que no se pueda llegar a la solución completa, se retrocede y se prueba con otras combinaciones. Este método suele resultar ineficiente debido a su naturaleza exhaustiva que hace que se exploren todas las soluciones posibles.

Cabe destacar que esta estrategia se ha implementado de dos maneras diferentes, una primera sin el vector "usadas" y otra con este. Este elemento nos proporciona una alternativa a la implementación de la función "usadas()", que es muy lenta debido a que repite muchas comprobaciones. La solución a este problema mediante el vector usadas se basa en usar un array que indique las tareas que están ya usadas en la asignación actual, de esta forma cada elemento del vector "usada[i]" indica cuántas veces la tarea "i" a la que hace referencia ha sido utilizada en la planificación actual. Si está usada sólo una vez, la asignación es válida, pero si está usada más de una vez, no.

2-Explicación componentes de s

Para representar los valores de las soluciones ya sean parciales o finales, dependiendo del nivel en el que se encuentre el algoritmo, empleamos el vector s, en el cual se van guardando las asignaciones de las soluciones en cada uno de los niveles. La variable 's' en este código representa la asignación de tareas a personas.

3-Funciones

La función de asignación principal en este algoritmo dentro de nuestra implementación será la función "generar()" asigna la tarea correspondiente a cada nivel(cada nivel equivale a una tarea) a un individuo representado en cada elemento del array "s".

También cabe destacar que se produce una asignación en la función “Retroceder()” al asignar nuevos valores para un elemento del vector “s”, o lo que es lo mismo una persona a la que se le asigna una tarea, y retroceder un nivel, es decir regresar en la búsqueda volviendo a la tarea anterior.

4-Presentación y análisis de resultados de número de pasos en las funciones sin usadas y con usadas

Durante la ejecución del código, medimos el número de nodos visitados y los pasos necesarios para cada una de las funciones de “Criterio()”, “Generar()”, “Solución()”, “MásHermanos()” y “Retroceder()”. Los resultados obtenidos nos sirven para obtener la siguientes tabla, la cuales nos permitirán sacar conclusiones acerca de la complejidad de la tarea y el impacto que tiene el uso o no del vector usadas en lugar de la implementación original de la función “Criterio()”.

n	Vector Usadas (Sí/No)	Nº nodos visitados	Nº pasos Criterio	Nº pasos Generar	Nº pasos Solución	Nº pasos MasHnos	Nº pasos Retroc
3	No	16	138	30	30	40	10
3	Sí	16	48	30	30	40	10
6	No	1957	74892	7422	7422	8659	1237
6	Sí	1957	11742	7422	7422	8659	1237

Tabla 1: resultados de “backtracking” (Alejandro)

n	VectorUsadas (Sí/No)	Nº nodos visitados	Nº pasos Criterio	Nº pasos Generar	Nº pasos Solución	Nº pasos MasHnos	Nº pasos Retroc
3	No	16	69	30	30	40	10
3	Sí	16	48	30	30	40	10
6	No	1957	37446	7422	7422	8659	1237
6	Sí	1957	11742	7422	7422	8659	1237

Tabla 2: resultados de “backtracking” (Marcelo)

En primer lugar, cabe mencionar que las primeras conclusiones teóricas que sacamos acerca de la complejidad computacional del programa es que estamos ante una complejidad del orden de $O(n^n)$, o lo que es lo mismo, complejidad exponencial. Esto se debe a que se generar n nodos para que nivel del árbol, habiendo n niveles.

Tras analizar los datos obtenidos, observamos que los resultados cuadran con los esperados teóricamente, creciendo el número nodos visitados exponencialmente según se aumenta el tamaño de la matriz, y con ello los niveles del árbol.

Las mediciones realizadas sobre los pasos que se necesitan en cada una de las funciones también crecen exponencialmente según se aumenta el tamaño de la matriz.

Por último, es relevante mencionar el impacto que genera la substitución de la implementación de la función “Criterio()” por una en la que se incluye el vector usadas, lo cual ahorra comprobaciones y hace la función más rápida. Este cambio hace que se reduzcan en gran medida el número de pasos necesario para ver si una solución cumple el criterio en cualquiera de los casos con diferente tamaño de matriz (3 y 6), como se puede apreciar en los resultados prácticos.